Task 2 – Signal Equalizer

Introduction: Signal equalizer is a basic tool in the music and speech industry. It also serves in several biomedical applications like hearing aid abnormalities detection.

Description: Develop a web application that can open a signal and allow the user to change the magnitude of some frequency components via sliders then reconstruct back the signal. The application should have the following features.

- **Generic Mode:** where the user can divide the frequency range into arbitrary number of subdivisions and scale the frequencies in each of them by a specific scale (from 0 to 2). One method to do this is to allow the user to add the control of each subdivision one by one, e.g. via UI elements, the user can add a subdivision, control its location and width on the frequency axis, then control its scale. Then, s/he can add the next subdivision, etc. After finishing, this scheme can be saved in some setting file that can be re-opened easily later.

To validate your work, each group should prepare a synthetic signal file. The signal is an artificial signal that you should prepare, composed of a summation of several pure single frequencies across all the frequency range. This should help in tracking what happens to each frequency when an equalizer action is taken and whether the program behaves correctly or not.

- **Customized Modes:**
  1. **Musical Instruments Mode:** where each slider can control the magnitude of a specific musical instrument in the input music signal that is a mixture of at least four different musical instruments.
  2. **Animal Sounds Mode:** where each slider can control the magnitude of a specific animal sound in a mixture of at least four animal sounds.
  3. **Human Voices Mode:** where each slider can control the magnitude of a specific person in a mixture of at least four different persons voices of different combinations of any of the following: males/females, young/old, different languages).
  4. **ECG Abnormalities Mode:** You need to search for and collect 4 ECG signals, one normal and each of the other three has a specific type of arrythmia. Then, each slider can control the magnitude of the arrythmia component in the input signal.

  Note that for all the customized modes, each slider does not necessarily correspond to one continuous range of frequencies. Each slider can easily correspond to multiple frequency ranges/windows. These schemes should be saved in a setting file that can be edited outside the software, and loaded again inside the software. Upon loading the setting file, all the controls should be created automatically.
- The user can switch between the modes easily (probably through an option menu or combobox). The UI is not expected to change dramatically among the different modes. The main expected change in UI when changing from one mode to the other is the change in the sliders captions and maybe the number of sliders.
- For all modes, the graph that displays the Fourier transform of the signal should have the flexibility to show the frequency range in either a linear scale or the Audiogram scale (Please, make your research for what Audiogram is). The user can switch between the two scales via some UI without interrupting or resetting any functionality.
- Beside the equalizer sliders, the UI should contain:
  - Two linked cine signal viewers, one for the input and one for the output signals, with a full set of functionality panel (play/stop/pause/speed-control/zoom/pan/reset) that respects all the boundary conditions. The two viewers should allow the signals to run in time in a synchronous way (i.e. the two viewers have to be exactly linked. They should always show the same exact time-part of the signal if the user scrolls or zooms on any one of them).
    - Cine viewer: A viewer that allows the signals to run in time.
    - Linked viewers: All viewers should show the same exact view (in time and magnitude directions) of the displayed signal.
  - Ability to run any signal as sound that can be heard for relevant signals.

    - Two spectrograms, one for the input and one for the output signals. Upon any change in any of the equalizer sliders, the output spectrogram should reflect the performed changes.
        - There should be an option to toggle show/hide of the spectrograms.
- **Different wavelets:** Beside the regular frequency domain, you should allow the user to pick some different basis/wavelets other than the sin/cos associated with the Fourier transform, and do the same exact equalization functionality. For each of the above modes, do your research to determine which wavelet is more suitable/efficient and compare the obtained results using the standard frequency domain and these different wavelets.
- **AI Models:** For each of the above modes, make your search, and provide some pretrained AI model that performs the same required functionality. Compare the performance between your equalizer and the AI model.

Code practice:

1. **Avoid code repetition**! If you notice similar lines in your code with only a few numbers or variables different, then these lines are very good candidates for a "function"! Code repetition is a very bad practice and will be penalized. Note that this task has a lot of potential for code repetition if is the workflow was not well-designed.
2. Any "or" in the above requirements is for the user, and it is simply "And" for the developer.
3. You UI will be evaluated mostly on how easy to be used by users.
4. You should update the graph/results or any component that should be affected once you change any value of any ui component (drop menu/ text box ...) without need for a button to make changes!
5. You will be penalized for any of the following:
    a. bugs that happen during submission,
    b. unhandled edge cases,
    c. refreshing because app is stuck and so on,
    d. any code that is dummy/not used. Clean your code before submission.